My SQL

1. What is SQL?

SQL (**Structured Query Language**) is a programming language used to interact with databases.

- It is used to **store, retrieve, update, and delete data**.
- Example commands:
    - SELECT * FROM employees; → retrieves all employee records.
    - INSERT INTO students (id, name) VALUES (1, 'Lalit'); → adds a new student.

---

2. Structured Data

- Data that is organized in **rows and columns** (tables).
- Easy to store in relational databases.
- Examples:
    - Employee records (ID, Name, Salary).
    - Bank transactions.

---

3. Unstructured Data

- Data that does **not follow a fixed format**.
- Difficult to store in traditional databases, usually stored in NoSQL or data lakes.
- Examples:
    - Images, videos, audio files.
    - Social media posts.
    - Emails.

---

4. What is MySQL?

- MySQL is a **popular open-source Relational Database Management System (RDBMS)**.
- Uses SQL to manage data.
- Commonly used in web applications (e.g., WordPress, Facebook initially used MySQL).

---

5. What is a Database?

A **database** is a structured collection of data stored electronically.

- Acts like a **digital filing system**.
- Example: A library database storing book info (Title, Author, ISBN).

---

6. What is DBMS?

DBMS (**Database Management System**) is software that allows users to **create, read, update, and delete (CRUD)** data in a database.

- Examples: MySQL, Oracle, SQL Server, MongoDB.

---

7. Types of DBMS

1. **Hierarchical DBMS** – Data stored in a tree structure (like folders).
2. **Network DBMS** – Data stored as nodes and relationships (graph-like).
3. **Relational DBMS (RDBMS)** – Data stored in tables (rows & columns).
4. **Object-Oriented DBMS (OODBMS)** – Stores objects (data + methods).

---

8. What is Relational DBMS (RDBMS)?

- A DBMS that stores data in **tables** (relations).
- Uses **primary keys** (unique identifiers) and **foreign keys** (links between tables).
- Example:
    - **Customer Table** → customer_id, name.
    - **Orders Table** → order_id, customer_id, amount.
    - customer_id links both tables.

Examples of RDBMS: MySQL, PostgreSQL, Oracle, SQL Server.

---

9. Types of Modern Database Management Systems

Modern DBMS are designed to handle **big data, unstructured data, distributed systems, and cloud storage**. Some types are:

1. **Relational DBMS (SQL-based)** → MySQL, PostgreSQL, Oracle.
2. **NoSQL DBMS** (handles unstructured/semi-structured data) → MongoDB, Cassandra.
3. **In-Memory DBMS** (stores data in RAM for speed) → Redis, SAP HANA.
4. **Columnar DBMS** (optimized for analytics & big data) → Apache HBase, Amazon Redshift.
5. **Cloud Databases** → Google BigQuery, AWS RDS, Snowflake.
6. **Graph Databases** (focus on relationships) → Neo4j, Amazon Neptune.
7. **Time-Series Databases** (for time-stamped data like IoT, stock prices) → InfluxDB, TimescaleDB.

---

10. Schema & Instance

- **Schema** → Overall design/structure of the database (blueprint).
- **Instance** → Actual data present at a given time.

Example:
Schema = Table design
Instance = Records inside the table

---

11. Keys in DBMS

Primary Key

Unique + Not Null
Example: employee_id

Foreign Key

Establish a relationship between two tables.

Candidate Key

Multiple unique columns → any one can be PK.

Composite Key

PK is made of two or more columns.

Super Key

Any set of attributes that uniquely identifies a row.

---

12. ER Model (Entity-Relationship)

Components:

- **Entity** (Student, Employee)
- **Attributes** (Name, Age, Salary)
- **Relationship** (Works In, Belongs To)
- **Keys** (PK)

Relationship types:

- 1:1
- 1:M
- M: M

---

13. Transaction

A sequence of operations performed as a single unit.

Example:

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;
```

---

14. Concurrency Control

Multiple users are accessing data at the same time.

Techniques:

- Locking (Shared/Exclusive locks)
- Timestamp ordering
- Multiversion Concurrency Control (MVCC)

---

15. Data Independence

Physical Data Independence

Changing storage does not affect the schema.

Logical Data Independence

Changing the schema does not affect the application.

13. Natural Key

A **Natural Key** is a key that comes from **actual business data** and uniquely identifies a row.

It is not generated by the system.

📝 **Examples of Natural Keys**

1. Email Address

Emails are unique for each user.

user_id	email	name
1	john@gmail.com	John
2	sara@yahoo.com	Sara

Here, **email** can be a natural key.

ACID PROPERTIES

When we perform **transactions** in a database (INSERT, UPDATE, DELETE), the database must ensure reliability. ACID properties guarantee that **data remains correct, consistent, and safe** even during failures.

ACID stands for:

1️⃣ **A — Atomicity**
2️⃣ **C — Consistency**
3️⃣ **I — Isolation**
4️⃣ **D — Durability**

Let's understand each one.

1️⃣ Atomicity (All or Nothing)

A transaction must be **fully completed**, or **not done at all**.

Example:

Suppose you are transferring ₹1000 from Account A to Account B.

Transaction contains:

1. Deduct 1000 from A

2. Add 1000 to B

If **step 1 succeeds** but **step 2 fails**, the database must **rollback** step 1.

SQL Example:

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 1000 WHERE account_id = 1;

UPDATE accounts SET balance = balance + 1000 WHERE account_id = 2;

COMMIT;
```

If any query fails, → **ROLLBACK** happens.
That is **Atomicity**.

2️⃣ Consistency (Valid State → Valid State)

After the transaction, the database must move from one **valid state** to another **valid state**.

- No rules should be broken.
- Constraints should not be violated (Primary Key, Foreign Key, CHECK).

Example:

Account balance must not become negative (CHECK constraint).
If a transaction tries to set balance = -500, it will be rejected.

This ensures the database remains **valid and correct**.

3️⃣ Isolation (Transactions Don't Disturb Each Other)

Multiple users can run transactions at the same time.
Isolation ensures that **one transaction does not affect another** until it is complete.

Example:

User A: transferring money
User B: checking balance

User B should not see incomplete/half-updated data.

SQL provides different isolation levels:

- READ UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE (highest)

Example problem if isolation is not maintained:

- Dirty Reads
- Non-repeatable Reads
- Phantom Reads

Isolation prevents these issues.

4️⃣ Durability (Permanent Changes)

Once a transaction is **committed**, data is permanently stored—even if:

- system crashes
- server restarts
- power goes off

The database stores the committed data in:

- logs
- disk
- recovery files

So the data **will not be lost**.

Example: After a bank transfer is committed, money cannot disappear after a crash.

⭐ Summary Table

Property	Meaning	Simple Explanation
Atomicity	All or nothing	If one step fails, the whole transaction fails
Consistency	Valid → Valid	No rules/constraints should be broken
Isolation	Independent execution	Transactions don't interfere
Durability	Permanent	Data is safe after commit

NORMALIZATION

🧠 What is Normalization?

Normalization is the process of **structuring a relational database** to minimize **data redundancy** and **improve data integrity**.

Goals:

- ✅ Store each fact only once
- ✅ Prevent data anomalies (insert/update/delete errors)
- ✅ Make the database easier to maintain and query

It's done in **steps called "Normal Forms" (NF).**

🛑 Why Normalization is Important

Imagine this single **Orders** table:

OrderID	CustomerName	CustomerPhone	Product1	Product2	Product3
101	John	9998887777	iPhone	Laptop	NULL

Problems

- **Repetition:** Customer name/phone repeats for each order
- **Inconsistent data:** John's phone might change in one row but not others
- **Difficulty adding data:** Can't add a new customer without an order
- **Difficulty deleting data:** Deleting an order may also delete customer info

⚡ The 6 Main Normal Forms

Each normal form is a step toward a clean, consistent design.

1️⃣ First Normal Form (1NF)

Rule:

- Each column must hold **atomic values** (no multiple values in one column).
- Each record must be **unique**.

❌ Bad Table

OrderID	CustomerName	Products
101	John	iPhone, Laptop

👉 The Products column has multiple values.

✅ 1NF Table

OrderID	CustomerName	Product
101	John	iPhone
101	John	Laptop

Now each cell is **atomic**.

2️⃣ Second Normal Form (2NF)

Rule:

- Be in 1NF.
- Remove **partial dependency** (when a non-key column depends only on part of a composite primary key).

👉 Happens only if the table's **primary key has more than one column**.

❌ Bad Table

OrderID	ProductID	CustomerName
101	P1	John
101	P2	John

- Primary Key = (OrderID, ProductID)
- CustomerName depends only on OrderID (not on full key)

✅ 2NF Solution

Orders

OrderID	CustomerName
101	John

OrderDetails

OrderID	ProductID
101	P1
101	P2

- Now each non-key attribute depends on the **whole primary key**.

3️⃣ Third Normal Form (3NF)

Rule:

- Be in 2NF.
- Remove **transitive dependency** (when a non-key column depends on another non-key column).

❌ Bad Table

CustomerID	CustomerName	ZipCode	City
1	John	12345	NY

- City depends on ZipCode, not directly on CustomerID.

✅ 3NF Solution

Customer

CustomerID	CustomerName	ZipCode
1	John	12345

ZipCodeInfo

ZipCode	City
12345	NY

- Each non-key attribute now depends **only on the primary key**.

4️⃣ Boyce–Codd Normal Form (BCNF)

Rule:

- Be in 3NF.
- Every determinant (attribute that defines another attribute) must be a candidate key.

👉 Rare case, but happens when:

- A table has **multiple candidate keys**.
- A non-key attribute determines part of a key.

Example:

Course	Teacher
SQL	Asha
Python	Rahul
SQL	Asha

Rules:

- A **teacher teaches only one course** → Teacher → Course
- A course can have multiple teachers → Course → Teacher

Two functional dependencies:

- Course → Teacher
- Teacher → Course

But **neither is a super key**, so BCNF is violated.

✓ Convert to BCNF

Teachers_Course

Teacher	Course
Asha	SQL
Rahul	Python

Now each determinant is a key.

5️⃣ Fourth Normal Form (4NF)

Rule:

- Be in BCNF.
- No **multi-valued dependencies**.

❌ **Bad Table**

Student	Language	Sport
John	English	Football
John	French	Cricket

- Two independent many-to-many relationships stored together.

✅ **4NF Solution**

StudentLanguage

Student	Language
John	English
John	French

StudentSport

Student	Sport
John	Football
John	Cricket

6️⃣ **Fifth Normal Form (5NF)**

Rule:

- Be in 4NF.
- No **join dependency** (data should not need multiple joins to reconstruct original info).
- Used in very complex business scenarios.
- Rare for typical apps.

🛡️ **Denormalization (Reverse Process)**

- Sometimes, we **intentionally break normalization** for faster reads.
- Example: Creating a SalesSummary table to avoid joining multiple tables repeatedly.

⚠️ Trade-offs:

- Pros: Faster reporting, simpler queries.
- Cons: Data duplication, possible inconsistency when updating.

🛠️ **Steps to Normalize a Database (Practical)**

1. **List all fields & data points you need** (e.g., Customer name, address, orders, products).
2. **Combine logically related data into one table**.
3. Apply **1NF**:
    - Remove repeating groups and nested data.
4. Apply **2NF**:

- If composite keys exist, move columns that don't depend on the full key.

5. Apply **3NF**:
   - Remove fields dependent on non-key attributes.
6. Optionally check **BCNF** if there are multiple candidate keys.
7. Test with sample queries — check if you can still retrieve everything easily.

🔥 **Real Example — E-commerce Normalization**

Step 1 — Raw Table

OrderID	OrderDate	CustomerName	CustomerEmail	ProductName	Price
101	20-09-2025	John	john@x.com	iPhone	700
101	20-09-2025	John	john@x.com	AirPods	200

Step 2 — 1NF (Atomic values)

➡️ Already atomic here.

Step 3 — 2NF (Separate Customers)

Customer

CustomerID	Name	Email
1	John	john@x.com

Order

OrderID	OrderDate	CustomerID
101	20-09-2025	1

OrderDetails

OrderID	ProductName	Price
101	iPhone	700
101	AirPods	200

Step 4 — 3NF (Separate Products)

Product

ProductID	ProductName	Price
P1	iPhone	700
P2	AirPods	200

OrderDetails

OrderID	ProductID
101	P1
and	P2

Now the database is clean, no redundancy, and easy to update.

DATA TYPES

1. Numeric Data Types

Used to store numbers.

🔹 Integer Types

Data Type	Storage	Range (Signed)	Example
TINYINT	1 byte	-128 to 127	Age (0–120)
SMALLINT	2 bytes	-32,768 to 32,767	Small counters
MEDIUMINT	3 bytes	-8M to 8M	Larger counts
INT / INTEGER	4 bytes	-2B to 2B	IDs, Salary
BIGINT	8 bytes	Very large numbers	Population, bank balance

👉 Use UNSIGNED if you only want positive values.
Example: INT UNSIGNED → 0 to 4,294,967,295.

🔹 Decimal & Floating Types

Data Type	Usage	Example
DECIMAL(M, D)	Exact fixed-point numbers (M = total digits, D = digits after decimal)	DECIMAL(8,2) → 123456.78
FLOAT	Approximate numbers (4 bytes)	3.14, 123.45
DOUBLE / REAL	Larger approximate numbers (8 bytes)	Scientific values

👉 Use DECIMAL for money (precise), FLOAT/DOUBLE for scientific/approximate values.

---

2. String Data Types

Used to store text or characters.

Data Type	Description	Example
CHAR(n)	Fixed length (padded with spaces). Faster for fixed size.	CHAR(5) → "Lalit"
VARCHAR(n)	Variable length (saves space). Most common for text.	VARCHAR(50) → "OpenAI"

TEXT	Large text data (up to 65,535 chars).	Blogs, descriptions
TINYTEXT	Up to 255 chars	Small notes
MEDIUMTEXT	Up to 16M chars	Articles
LONGTEXT	Up to 4GB text	Books, logs
BLOB (Binary Large Object)	Stores binary data (images, videos).	Photo, PDF
ENUM	Predefined list of values.	Gender: ENUM('Male','Female')
SET	Multiple predefined values (like checkboxes).	SET('Read','Write','Execute')

---

3. Date & Time Data Types

Used to store temporal values.

Data Type	Format	Example
DATE	YYYY-MM-DD	26-09-2025
DATETIME	YYYY-MM-DD HH:MM: SS	26-09-2025 11:30
TIMESTAMP	Same as DATETIME, but auto-updates with the current time	26-09-2025 11:30
TIME	HH:MM: SS	14:20:00
YEAR	YYYY	2025

✅ **Summary:**

- **Numbers** → INT, DECIMAL, FLOAT, DOUBLE.
- **Text** → CHAR, VARCHAR, TEXT, BLOB, ENUM, SET.
- **Dates** → DATE, DATETIME, TIMESTAMP, TIME, YEAR.
- **Modern** → JSON.

SQL COMMANDS

1. DDL (DATA DEFINITION LANGUAGE)

DDL (Data Definition Language) commands in MySQL are used to **define, modify, and manage the structure** of database objects — such as tables, schemas, indexes, and views.

These commands affect the **structure** of the database, not the data inside it.

🔹 Common DDL Commands in MySQL

Command	Description	Example
CREATE	Used to create new database objects like tables, databases, or views.	CREATE TABLE students (id INT, name VARCHAR(50), age INT);
ALTER	Used to modify the structure of an existing table (add, delete, or modify columns).	ALTER TABLE students ADD COLUMN email VARCHAR(100);
DROP	Deletes an entire database or table permanently.	DROP TABLE students;
TRUNCATE	Removes all data from a table but keeps its structure.	TRUNCATE TABLE students;
RENAME	Changes the name of a table.	RENAME TABLE students TO learners;

⚙️ Key Points

- DDL commands **auto-commit** in MySQL — meaning changes **cannot be rolled back** once executed.
- They define **how the data is stored**, not the data itself.
- DDL affects the **database schema**, while DML (Data Manipulation Language) affects **table data**

🧠 Example Flow

🟢 1. CREATE

Used to create a new **database** or **table**.

```
-- Create a new database
CREATE DATABASE school;

-- Use that database
USE school;

-- Create a new table
CREATE TABLE students (
   student_id INT PRIMARY KEY,
   name VARCHAR(50),
   age INT,
   grade VARCHAR(10)
);
```

✅ **Result:** A table named students is created with 4 columns.

🟡 **2. ALTER**

Used to **modify the structure** of an existing table.

```
-- Add a new column
ALTER TABLE students ADD COLUMN email VARCHAR(100);

-- Modify a column data type
ALTER TABLE students MODIFY COLUMN grade VARCHAR(20);

-- Rename a column
ALTER TABLE students CHANGE grade class VARCHAR(20);

-- Drop (delete) a column
ALTER TABLE students DROP COLUMN age;
```

✅ **Result:** The table structure changes — a new column added, type modified, and one removed.

🔵 **3. RENAME**

Used to **change the table name**.

```
RENAME TABLE students TO school_students;
```

✅ **Result:** Table students is now called school_students.

🔴 **4. TRUNCATE**

Used to **remove all records** from a table but **keep its structure**.

```
TRUNCATE TABLE school_students;
```

✅ **Result:** All rows deleted — table stays empty, structure remains.

⚫ **5. DROP**

Used to **delete the table or database permanently**.

```
-- Delete a table
DROP TABLE school_students;

-- Delete a database
DROP DATABASE school;
```

✅ **Result:** The table or database is permanently removed — cannot be recovered.

---

2. DML (DATA MANIPULATION LANGUAGE)

DML (Data Manipulation Language) deals with the **data stored in tables** — it lets you **insert**, **update**, **delete**, and **retrieve** records.

🧱 Common DML Commands

Command	Description	Example
INSERT	Add new records (rows) into a table.	INSERT INTO employees VALUES (1, 'John', 'HR', 50000);
UPDATE	Modify existing records in a table.	UPDATE employees SET salary = 60000 WHERE emp_id = 1;
DELETE	Remove records from a table.	DELETE FROM employees WHERE emp_id = 1;
SELECT	Retrieve data from a table.	SELECT * FROM employees;

🏢 Example: Using DML on an Employees Table

Let's use the table we created earlier 👇

```
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(50),
  dept_name VARCHAR(50),
  salary DECIMAL(10,2)
);
```

🟢 1. INSERT — Add New Data

```
INSERT INTO employees (emp_id, emp_name, dept_name, salary)
VALUES
(1, 'John', 'HR', 50000),
(2, 'Sara', 'IT', 65000),
(3, 'Raj', 'Finance', 55000);
```

✅ **Adds 3 rows** to the employees table.

🟡 2. SELECT — View the Data

```
SELECT * FROM employees;
```

✅ **Displays** all rows and columns in the table.

Example Output:

emp_id	emp_name	dept_name	salary
1	John	HR	50000
2	Sara	IT	65000
3	Raj	Finance	55000

🔵 **3. UPDATE — Modify Data**

```
UPDATE employees
SET salary = 70000
WHERE emp_id = 2;
```

✅ **Updates** Sara's salary to 70000.

🔴 **4. DELETE — Remove Data**

```
DELETE FROM employees
WHERE emp_id = 3;
```

✅ **Deletes** the record of Raj.

⚫ **5. SELECT (with condition)**

To see only employees in a certain department:

```
SELECT emp_name, salary
FROM employees
WHERE dept_name = 'IT';
```

✅ **Displays only** IT department employees.

🧠 **Key Points to Remember**

- DML commands **affect table data**, not the structure.
- Changes made by DML can be **rolled back** (undone) if used with transactions.
- DML commands are **most commonly used** in day-to-day database operations.

3. DCL

DCL (Data Control Language) commands are used to **control access and permissions** to the data stored in a database.
They define **who can do what** — for example, who can read, insert, update, or delete data from tables.

Command	Description	Example
GRANT	Gives specific privileges (permissions) to a user.	GRANT SELECT, INSERT ON company.* TO 'lalit'@'localhost';
REVOKE	Removes or cancels privileges given to a user.	REVOKE INSERT ON company.* FROM 'lalit'@'localhost';

🏢 **Example: DCL in Action**

Let's assume you have a database named company and a user named **lalit**.

🟢 1. GRANT — Give Permissions

```
GRANT SELECT, INSERT, UPDATE
ON company.*
TO 'lalit'@'localhost'
IDENTIFIED BY 'password123';
```

✅ **Meaning:**
User **lalit** can:

- View data (SELECT)
- Add new data (INSERT)
- Modify existing data (UPDATE)
  in **all tables** of the company database.

🔴 2. REVOKE — Remove Permissions

```
REVOKE UPDATE
ON company.*
FROM 'lalit'@'localhost';
```

✅ **Meaning:**
User **lalit** can no longer perform UPDATE operations in the company database.

⚙️ 3. SHOW GRANTS — Check Permissions

```
SHOW GRANTS FOR 'lalit'@'localhost';
```

✅ **Displays** all permissions currently assigned to user **lalit**.

🧠 Key Points About DCL

- **Controls user access** and database security.
- Usually used by **Database Administrators (DBAs)**.
- DCL commands **require privileges** — not every user can grant or revoke access.
- Changes made with DCL **affect database security**, not the data itself.

Command	Purpose	Example
GRANT	Give the user permission	GRANT SELECT ON school.* TO 'user1'@'localhost';
REVOKE	Take away permission	REVOKE SELECT ON school.* FROM 'user1'@'localhost';

4. TCL (Transaction Control Language)

TCL (Transaction Control Language) commands are used to **manage transactions** in a database.
A **transaction** is a set of SQL operations that are executed as a **single unit of work** — meaning *either all of them succeed, or none do*.
These commands help ensure **data integrity** and allow you to **undo or confirm changes** made by DML commands (INSERT, UPDATE, DELETE).

Command	Description	Example
COMMIT	Saves all changes made during the current transaction permanently.	COMMIT;
ROLLBACK	Cancels all changes made in the current transaction.	ROLLBACK;
SAVEPOINT	Sets a point within a transaction to which you can roll back.	SAVEPOINT sp1;
RELEASE SAVEPOINT	Removes a savepoint.	RELEASE SAVEPOINT sp1;
SET AUTOCOMMIT	Enables or disables automatic commit of changes.	SET AUTOCOMMIT = 0;

🏢 **Example: Using TCL with DML Commands**
🏦 **Example: Money Transfer Between Two Accounts**
Let's say we have a table called accounts:

```
CREATE TABLE accounts (
   acc_no INT PRIMARY KEY,
   acc_name VARCHAR(50),
   balance DECIMAL(10,2)
);
```

Now, suppose we have two accounts:

acc_no	acc_name	balance
101	Lalit	10,000.00
102	Ravi	5,000.00

🎯 **Goal:**
Transfer ₹2000 from **Lalit's account (101)** to **Ravi's account (102)** safely — either **both operations succeed or both fail** (no partial change).

⚙️ **Step-by-Step Using TCL Commands**

🔹 **Step 1: Start the Transaction**

```
SET AUTOCOMMIT = 0;      -- Disable auto-commit
START TRANSACTION;
```

✅ This ensures MySQL doesn't save changes automatically —
you'll decide when to COMMIT or ROLLBACK.

🔹 **Step 2: Perform the Transfer Operations**

1️⃣ **Deduct ₹2000 from Lalit's account**

```
UPDATE accounts
SET balance = balance - 2000
WHERE acc_no = 101;
```

2️⃣ **Add ₹2000 to Ravi's account**

```
UPDATE accounts
SET balance = balance + 2000
WHERE acc_no = 102;
```

✅ Both updates are part of the same transaction.

🔹 **Step 3: Verify the Result (Optional)**

```
SELECT * FROM accounts;
```

You'll see:

acc_no	acc_name	balance
101	Lalit	8000
102	Ravi	7000

✅ Looks correct so far — but not yet saved!

🔹 **Step 4: If Everything is Correct → COMMIT**

```
COMMIT;
```

✅ This **permanently saves** both updates in the database.
Now the transfer is **successful and permanent**.

🔹 **Step 5: If Something Goes Wrong → ROLLBACK**

Suppose there was an error (e.g., Ravi's account doesn't exist):

```
ROLLBACK;
```

✅ This **undoes all changes** made since the transaction started —
so Lalit's ₹2000 is restored, and no money is lost.

🧠 **Why TCL is Important**

Benefit	Description
Data Consistency	Ensures all related operations succeed or fail together.
Error Handling	You can undo changes if something goes wrong.
Safe Transactions	Especially useful for financial, e-commerce, or booking systems.

✅ **Final Table After Commit**

acc_no	acc_name	balance
101	Lalit	8000
102	Ravi	7000

🏦 **Example: Bank Transfer Using SAVEPOINT**

We'll use the same table:

```
CREATE TABLE accounts (
   acc_no INT PRIMARY KEY,
   acc_name VARCHAR(50),
   balance DECIMAL(10,2)
);
```

Sample Data:

acc_no	acc_name	balance
101	Lalit	10,000.00
102	Ravi	5,000.00
103	Meena	8,000.00

🎯 **Scenario:**

We will perform multiple transactions:

1. Transfer ₹2000 from **Lalit → Ravi**
2. Transfer ₹1000 from **Lalit → Meena**
3. Then, decide to **undo only the second transfer** (Lalit → Meena), not both.

⚙️ **Step-by-Step Using TCL + SAVEPOINT**

🔹 **Step 1: Start the Transaction**

```
SET AUTOCOMMIT = 0;
START TRANSACTION;
```

✅ This turns off auto-saving — so you can control commits manually.

🔹 **Step 2: First Transfer (Lalit → Ravi)**

```
UPDATE accounts SET balance = balance - 2000 WHERE acc_no = 101;
UPDATE accounts SET balance = balance + 2000 WHERE acc_no = 102;
```

✅ Lalit's balance = 8000
✅ Ravi's balance = 7000
Now set a savepoint here 👇

```
SAVEPOINT sp1;
```

✅ sp1 marks this safe point — you can return here later if needed.

🔹 **Step 3: Second Transfer (Lalit → Meena)**

```
UPDATE accounts SET balance = balance - 1000 WHERE acc_no = 101;
UPDATE accounts SET balance = balance + 1000 WHERE acc_no = 103;
```

✅ Lalit's balance = 7000
✅ Meena's balance = 9000

🔹 **Step 4: Realize a Mistake** 😅
Oops! You didn't mean to transfer to Meena.
So, you roll back **only** to the first savepoint.

```
ROLLBACK TO sp1;
```

✅ All changes **after** sp1 (the second transfer) are undone.
✅ Lalit's balance returns to 8000; Meena's returns to 8000.

🔹 **Step 5: Commit Final Correct Changes**

```
COMMIT;
```

✅ Now only the **first transfer (Lalit → Ravi)** is saved permanently.
✅ The second one (Lalit → Meena) is canceled safely.

✅ **Final Balances After COMMIT**

acc_no	acc_name	balance
101	Lalit	8000
102	Ravi	7000
103	Meena	8000

🧠 Key Takeaways

TCL Command	Purpose
SAVEPOINT sp1;	Marks a specific point inside a transaction.
ROLLBACK TO sp1;	Undoes all changes made after that point.
COMMIT;	Saves all remaining changes permanently.
SET AUTOCOMMIT = 0;	Disables automatic saving so you can control transactions manually.

5. DQL (DATA QUERY LANGUAGE)

DQL (Data Query Language) is used to **fetch or query data** from a database.
It helps you retrieve specific information stored in tables — for example, all employee names, total sales, or customers from a certain city.
In MySQL, **DQL mainly consists of the SELECT statement**.

🧱 DQL Command in MySQL

Command	Description	Example
SELECT	Used to retrieve data from one or more tables.	SELECT * FROM employees;

That's it — DQL focuses **only on querying data**, not modifying it.

🏢 **Example: Using DQL (SELECT)**
Suppose we have a table called employees:

emp_id	emp_name	dept_name	salary
1	John	HR	50000
2	Sara	IT	65000
3	Raj	Finance	55000
4	Meena	IT	60000

🟢 **1. Retrieve All Columns**

```
SELECT * FROM employees;
```

✅ Shows all rows and all columns.

🟡 **2. Retrieve Specific Columns**

```
SELECT emp_name, salary FROM employees;
```

✅ Shows only employee names and their salaries.

🔵 **3. Use WHERE to Filter Data**

```
SELECT * FROM employees WHERE dept_name = 'IT';
```

✅ Displays only IT department employees.

emp_name	dept_name	salary
Sara	IT	65000
Meena	IT	60000

🟣 **4. Use ORDER BY to Sort Data**

```
SELECT emp_name, salary FROM employees ORDER BY salary DESC;
```

✅ Displays employees in **descending order** of salary.

🟠 **5. Use Aggregate Functions**

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_name;
```

✅ Shows **average salary** by department.

dept_name	avg_salary
HR	50000
IT	62500
Finance	55000

🔴 **6. Use LIMIT to Restrict Rows**

```
SELECT * FROM employees LIMIT 2;
```

✅ Returns only the first 2 records.

🧠 **Key Points**

Feature	Description
DQL full form	Data Query Language
Main command	SELECT
Purpose	Retrieve data from database tables
Affects data?	❌ No — it only reads data
Rollback/Commit?	Not needed — no data changes

CONSTRAINTS

🔹 **What are Constraints in MySQL?**

Constraints are **rules** applied to table columns to **maintain data accuracy, integrity, and reliability**.
They ensure that **invalid or inconsistent data** cannot be entered into a table.
Think of constraints as **guardrails** that protect your data.

🧱 **Types of Constraints in MySQL**

Constraint	Description	Example
NOT NULL	Ensures that a column cannot have NULL values.	name VARCHAR(50) NOT NULL
UNIQUE	Ensures all values in a column are different.	email VARCHAR(100) UNIQUE
PRIMARY KEY	Uniquely identifies each record (combines NOT NULL + UNIQUE).	emp_id INT PRIMARY KEY
FOREIGN KEY	Links two tables together (maintains referential integrity).	FOREIGN KEY (dept_id) REFERENCES department(dept_id)
CHECK	Ensures values meet a specific condition.	CHECK (salary > 0)
DEFAULT	Sets a default value if no value is given.	status VARCHAR(10) DEFAULT 'Active'

🏢 **Example: Constraints in Action**

Let's create a table with different constraints 👇

```
CREATE TABLE employees (
   emp_id INT PRIMARY KEY,                -- Unique & not null
   emp_name VARCHAR(50) NOT NULL,         -- Cannot be null
   email VARCHAR(100) UNIQUE,             -- No duplicate emails
   salary DECIMAL(10,2) CHECK (salary > 0),   -- Salary must be positive
   dept_id INT,
   status VARCHAR(10) DEFAULT 'Active',       -- Default value if not given
   FOREIGN KEY (dept_id) REFERENCES department(dept_id) -- Links to another table
);
```

🧩 **Explanation of Each Constraint**

🔹 **1. NOT NULL**

- Ensures a column **cannot be left empty**.
-

```
emp_name VARCHAR(50) NOT NULL
```

✅ Every employee **must have a name**.

🔹 **2. UNIQUE**

- Ensures **no duplicate values** are entered in that column.
-

email VARCHAR(100) UNIQUE

✅ No two employees can have the same email address.

🔹 **3. PRIMARY KEY**

- Uniquely identifies each record.
- Automatically implies **NOT NULL + UNIQUE**.

emp_id INT PRIMARY KEY

✅ Each employee ID is unique and not empty.

🔹 **4. FOREIGN KEY**

- Connects one table to another (parent–child relationship).
-

FOREIGN KEY (dept_id) REFERENCES department(dept_id)

✅ Ensures each dept_id in employees must exist in the department table.

🔹 **5. CHECK**

- Ensures data meets a condition.

CHECK (salary > 0)

✅ Prevents inserting negative or zero salary.

🔹 **6. DEFAULT**

- Sets a value automatically if none is provided.

status VARCHAR(10) DEFAULT 'Active'

✅ If you don't specify a status, it becomes 'Active' by default.

🧠 **Key Points**

Concept	Meaning
Data Integrity	Constraints keep your data accurate and consistent.
Automatic Enforcement	MySQL enforces constraints automatically — you don't need to write extra logic.
Error Message	If you try to insert invalid data, MySQL will block it.

🧩 **Quick Example: Test the Rules**

```
INSERT INTO employees (emp_id, emp_name, salary)
VALUES (1, NULL, -5000);
```

❌ Error!

- emp_name cannot be NULL (NOT NULL fails)
- salary cannot be negative (CHECK fails)

✅ Correct Example:

```
INSERT INTO employees (emp_id, emp_name, salary)
VALUES (1, 'Lalit', 50000);
```

AGGREGATE FUNCTIONS

Aggregate functions in MySQL are used to **perform calculations on multiple rows** and **return a single value** as the result.
They are most commonly used with the **GROUP BY** clause to summarize data — for example, finding total sales, average salary, or highest marks in each group.

🧱 **Common Aggregate Functions in MySQL**

Function	Description	Example
COUNT()	Counts the number of rows (or non-null values).	COUNT(*), COUNT(column_name)
SUM()	Adds up all numeric values in a column.	SUM(salary)
AVG()	Returns the average (mean) value.	AVG(salary)
MIN()	Finds the smallest value in a column.	MIN(salary)
MAX()	Finds the largest value in a column.	MAX(salary)

🏢 **Example Table: employees**

emp_id	emp_name	dept_name	salary
1	John	HR	50000
2	Sara	IT	65000
3	Raj	IT	55000
4	Meena	HR	60000
5	Amit	Finance	70000

🟢 **1. COUNT() — Count Records**

```
SELECT COUNT(*) AS total_employees FROM employees;
```

✅ **Output:**

total_employees
5

Explanation: Counts all rows in the table.

🟡 **2. SUM() — Total of a Column**

```
SELECT SUM(salary) AS total_salary FROM employees;
```

✅ **Output:**

total_salary
300000

Explanation: Adds up all employee salaries.

🔵 3. AVG() — Average Value

```
SELECT AVG(salary) AS average_salary FROM employees;
```

✅ **Output:**

average_salary
60000

Explanation: Calculates the mean salary of all employees.

🟣 4. MIN() — Minimum Value

```
SELECT MIN(salary) AS lowest_salary FROM employees;
```

✅ **Output:**

lowest_salary
50000

Explanation: Finds the smallest salary.

🟠 5. MAX() — Maximum Value

```
SELECT MAX(salary) AS highest_salary FROM employees;
```

✅ **Output:**

highest_salary
70000

Explanation: Finds the largest salary.

🔹 Combine with GROUP BY

You can use aggregate functions **with GROUP BY** to summarize data per department:

```
SELECT dept_name,
    COUNT(*) AS total_employees,
    SUM(salary) AS total_salary,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_name;
```

✅ **Output:**

dept_name	total_employees	total_salary	avg_salary
HR	2	110000	55000
IT	2	120000	60000
Finance	1	70000	70000

🔹 **Use with HAVING Clause**

If you want to filter groups **after aggregation**, use HAVING (not WHERE):

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_name
HAVING AVG(salary) > 55000;
```

✅ **Output:**

dept_name	avg_salary
IT	60000
Finance	70000

🧠 **Key Points**

Concept	Explanation
Aggregate functions	Operate on multiple rows and return one result
Used with	Often used with GROUP BY
WHERE vs HAVING	WHERE filters before aggregation; HAVING filters after
Nulls	Most aggregate functions ignore NULL values

CLAUSES IN MYSQL

🔹 **What is a Clause in MySQL?**

A **clause** in MySQL is a **part of an SQL statement** that performs a specific task or defines a condition.
It helps you **filter, sort, group, or limit** data returned from a query.
👉 You can think of clauses as **components** that make up a full SQL command.

🧱 **Common Clauses in MySQL**

Here's a quick overview 👇

Clause	Used For	Appears In Query
SELECT	Choose the columns to display	Always starts the query
DISTINCT	Remove duplicate rows from the result	Right after SELECT
FROM	Specify the table to get data from	After SELECT (or DISTINCT)
WHERE	Filter rows based on conditions	Before GROUP BY
GROUP BY	Group rows that have the same values	After WHERE
HAVING	Filter the grouped results	After GROUP BY
ORDER BY	Sort the result set (ASC/DESC)	Near the end
LIMIT	Restrict the number of rows returned	Always last

🧠 **Example Query (showing order in action):**

```
SELECT DISTINCT dept_name, AVG(salary) AS avg_salary
FROM employees
WHERE salary > 50000
GROUP BY dept_name
HAVING AVG(salary) > 55000
ORDER BY avg_salary DESC
LIMIT 3;
```

⚙️ Step-by-Step Explanation

Step	Clause	What Happens
1️⃣	**FROM**	Choose the table (employees)
2️⃣	**WHERE**	Filter rows where salary > 50000
3️⃣	**GROUP BY**	Group rows by dept_name
4️⃣	**HAVING**	Keep only groups with an average salary > 55000
5️⃣	**SELECT / DISTINCT**	Display department and average salary, removing duplicates
6️⃣	**ORDER BY**	Sort results by avg_salary in descending order
7️⃣	**LIMIT**	Show only the top 3 rows

WILDCARD OPERATORS

These are super useful when you want to **search for patterns** in text data instead of exact matches.

🔹 **What are Wildcard Operators?**

Wildcard operators are **special symbols** used with the **LIKE** or **NOT LIKE** keyword in MySQL to perform **pattern matching** in text columns.

They help you **find values that are similar** to a given pattern — not necessarily an exact match.

🧱 **Syntax**

```
SELECT column_name
FROM table_name
WHERE column_name LIKE 'pattern';
```

or

```
SELECT column_name
FROM table_name
WHERE column_name NOT LIKE 'pattern';
```

🔸 **Common Wildcard Characters in MySQL**

Wildcard	Description	Example	Matches
%	Represents **zero, one, or many characters**	'A%'	Words starting with “A” (like “Amit”, “Anita”, “Apple”)
_ (underscore)	Represents **exactly one character**	'S_ra'	“Sara”, “Sira”, “Sora”, etc.
[ ]	Used to match **any single character within brackets** *(works in some MySQL modes)*	'[ABC]%'	Words starting with A, B, or C
[^]	Matches **any single character not within brackets** *(rarely used in MySQL)*	'[^A]%'	Words not starting with A

⚠️ Note: MySQL primarily supports **%** and **_**. The bracket syntax is more common in SQL Server or PostgreSQL.

🏢 **Example Table: employees**

emp_id	emp_name	dept_name
1	John	HR
2	Sara	IT
3	Sona	IT
4	Amit	Finance
5	Ankit	HR
6	Raj	Sales

🔹 1. Using % — Match Any Number of Characters

```
SELECT emp_name
FROM employees
WHERE emp_name LIKE 'A%';
```

✅ **Result:**

emp_name
Amit
Ankit

Explanation: Finds all names starting with **A**.

🔹 2. Using % in the Middle or End

```
SELECT emp_name
FROM employees
WHERE emp_name LIKE '%a%';
```

✅ **Result:**

emp_name
Sara
Sona
Raj
Ankit
Amit

Explanation: Finds names containing **'a** anywhere.

🔹 3. Using _ — Match a Single Character

```
SELECT emp_name
FROM employees
WHERE emp_name LIKE 'S_ra';
```

✅ **Result:**

emp_name
Sara

Explanation: The underscore (_) represents **exactly one letter**.

🔹 4. Using Multiple _ Characters

```
SELECT emp_name
FROM employees
WHERE emp_name LIKE '__na';
```

✅ **Result:**

emp_name
Sona

Explanation: Finds names with **4 letters**, ending in “na”.

🔹 5. Using NOT LIKE

```
SELECT emp_name
FROM employees
WHERE emp_name NOT LIKE 'A%';
```

✅ **Result:**

emp_name
John
Sara
Sona
Raj

Explanation: Excludes names that start with “A”.

🧠 **Summary Table**

Wildcard	Meaning	Example	Matches
%	Any number of characters	'A%'	Amit, Ankit
_	Exactly one character	'S_ra'	Sara
NOT LIKE	Exclude a pattern	NOT LIKE 'A%'	Names not starting with A

CASE WHEN STATEMENT

🔹 **What is a CASE WHEN Statement?**

The **CASE** statement in MySQL is used to perform **conditional logic** (similar to IF-THEN-ELSE) inside SQL queries.
It allows you to **apply conditions** and return **different values** based on those conditions.

🧱 **Basic Syntax**

```
CASE
   WHEN condition1 THEN result1
   WHEN condition2 THEN result2
   ...
   ELSE resultN
END
```

✅ **Explanation:**

- MySQL checks each condition **in order**.
- When it finds a **true** condition, it returns the corresponding **result**.
- If **no condition matches**, the **ELSE** part is executed.
- The END keyword marks the end of the CASE block.

🏢 **Example Table: employees**

emp_id	emp_name	dept_name	salary
1	John	HR	50000
2	Sara	IT	65000
3	Raj	IT	40000
4	Meena	HR	70000
5	Amit	Finance	30000

🔹 **1. Simple CASE Example (based on condition)**

```
SELECT emp_name,
      salary,
      CASE
         WHEN salary >= 60000 THEN 'High Salary'
         WHEN salary BETWEEN 40000 AND 59999 THEN 'Medium Salary'
         ELSE 'Low Salary'
      END AS salary_level
FROM employees;
```

✅ **Result:**

emp_name	salary	salary_level
John	50000	Medium Salary
Sara	65000	High Salary
Raj	40000	Medium Salary
Meena	70000	High Salary
Amit	30000	Low Salary

Explanation:
The CASE statement classifies each employee based on their salary range.

🔹 **2. CASE with GROUP BY (Example in Reports)**
You can use CASE inside aggregate functions to count category-based data.

```
SELECT
   dept_name,
   SUM(CASE WHEN salary >= 60000 THEN 1 ELSE 0 END) AS high_salary_count,
   SUM(CASE WHEN salary < 60000 THEN 1 ELSE 0 END) AS low_salary_count
FROM employees
GROUP BY dept_name;
```

✅ **Result:**

dept_name	high_salary_count	low_salary_count
HR	1	1
IT	1	1
Finance	0	1

Explanation:
Counts employees with high and low salaries in each department.

🔹 **3. CASE Used in ORDER BY**
You can also use CASE to define **custom sorting** rules.

```
SELECT emp_name, dept_name
FROM employees
ORDER BY
   CASE
      WHEN dept_name = 'IT' THEN 1
      WHEN dept_name = 'HR' THEN 2
      ELSE 3
   END;
```

✅ **Explanation:**

- IT department employees will appear first,
- HR next,
- and all others (like Finance) last.

🔹 4. CASE in UPDATE Statement

You can use CASE to update data conditionally.

```
UPDATE employees
SET salary =
  CASE
    WHEN dept_name = 'HR' THEN salary + 5000
    WHEN dept_name = 'IT' THEN salary + 3000
    ELSE salary
  END;
```

✅ **Explanation:**

- Adds ₹5000 to HR employee salaries,
- Adds ₹3000 to IT employee salaries,
- Leaves others unchanged.

🧠 **Key Points**

Concept	Description
Purpose	Perform conditional logic inside SQL
Similar To	IF–THEN–ELSE statement in programming
Usable In	SELECT, UPDATE, ORDER BY, GROUP BY
ELSE	Optional — used for default case
END	Must close every CASE expression

🧩 **Example (All-in-One)**

```
SELECT emp_name,
    dept_name,
    salary,
    CASE
      WHEN dept_name = 'IT' AND salary > 60000 THEN 'Senior IT Staff'
      WHEN dept_name = 'IT' THEN 'Junior IT Staff'
      WHEN dept_name = 'HR' THEN 'HR Staff'
      ELSE 'Other Dept'
    END AS designation
FROM employees;
```

✅ **Result:**

emp_name	dept_name	salary	designation
John	HR	50000	HR Staff
Sara	IT	65000	Senior IT Staff
Raj	IT	40000	Junior IT Staff
Meena	HR	70000	HR Staff
Amit	Finance	30000	Other Dept

JOINS

A **JOIN** is used to **combine rows from two or more tables** based on a **related column** between them — usually a **foreign key**.
It helps you fetch data that exists **across multiple tables**.

🧱 Basic Syntax

```
SELECT columns
FROM table1
JOIN table2
ON table1.common_column = table2.common_column;
```

🔸 Types of Joins in MySQL

Type	Description
INNER JOIN	Returns rows that have matching values in both tables
LEFT JOIN	Returns all rows from the left table + matching rows from the right table
RIGHT JOIN	Returns all rows from the right table + matching rows from the left table
FULL JOIN (simulated with UNION)	Returns all rows when there is a match in either table (shows unmatched rows from both sides as NULL)
CROSS JOIN	Returns all possible combinations (Cartesian product) — no join condition required
SELF JOIN	Joins a table with itself to compare rows within the same table (e.g., employee–manager relationship)

🏢 Example Tables

Table 1: employees

emp_id	emp_name	dept_id
1	John	101
2	Sara	102
3	Raj	103
4	Meena	101
5	Amit	NULL

Table 2: departments

dept_id	dept_name
101	HR
102	IT
103	Finance
104	Marketing

1. INNER JOIN
Returns only the records that have matching values in both tables.

```
SELECT employees.emp_name, departments.dept_name
FROM employees
INNER JOIN departments
ON employees.dept_id = departments.dept_id;
```

✅ **Result:**

emp_name	dept_name
John	HR
Sara	IT
Raj	Finance
Meena	HR

Explanation:
Amit is **not shown** because his dept_id is NULL and doesn't match any department.

---

2. LEFT JOIN (or LEFT OUTER JOIN)
Returns all rows from the left table, even if there's **no match** in the right table.
Missing values from the right table show as NULL.

```
SELECT employees.emp_name, departments.dept_name
FROM employees
LEFT JOIN departments
ON employees.dept_id = departments.dept_id;
```

✅ **Result:**

emp_name	dept_name
John	HR
Sara	IT
Raj	Finance
Meena	HR
Amit	NULL

Explanation:
Amit appears with NULL for dept_name because he doesn't belong to any department.

---

3. RIGHT JOIN (or RIGHT OUTER JOIN)
Returns all rows from the right table, even if there's **no match** in the left table.

```
SELECT employees.emp_name, departments.dept_name
FROM employees
RIGHT JOIN departments
ON employees.dept_id = departments.dept_id;
```

✅ **Result:**

emp_name	dept_name
John	HR
Sara	IT
Raj	Finance
Meena	HR
NULL	Marketing

Explanation:
Marketing appears even though no employee is assigned to it.

---

4. FULL JOIN (or FULL OUTER JOIN)
MySQL doesn't have a direct FULL JOIN,
but you can simulate it using a UNION of LEFT JOIN and RIGHT JOIN.

```
SELECT employees.emp_name, departments.dept_name
FROM employees
LEFT JOIN departments
ON employees.dept_id = departments.dept_id
UNION
SELECT employees.emp_name, departments.dept_name
FROM employees
RIGHT JOIN departments
ON employees.dept_id = departments.dept_id;
```

✅ **Result:**

emp_name	dept_name
John	HR
Sara	IT
Raj	Finance
Meena	HR
Amit	NULL
NULL	Marketing

Explanation:
Shows all employees and all departments — even unmatched ones.

---

5. CROSS JOIN
Returns all possible combinations (Cartesian product) of both tables.

```
SELECT employees.emp_name, departments.dept_name
FROM employees
CROSS JOIN departments;
```

✅ **Result:**
5 employees × 4 departments = 20 rows.
Explanation:
Every employee is paired with every department.

---

6. What is a SELF JOIN?

A **self-join** is when a table is **joined with itself**.
It's used when the **same table contains related data**, like a manager-employee relationship or parent-child structure.

👉 It's just a **regular join**, but the **table appears twice** in the query (using aliases).

🧱 Example Table — employees

emp_id	emp_name	manager_id
1	John	NULL
2	Sara	1
3	Raj	1
4	Meena	2
5	Amit	2

🔹 Problem

We want to know:

👉 Each employee's **manager name** (both stored in the same table).

✅ SELF JOIN Query

```
SELECT
   e.emp_name AS Employee,
   m.emp_name AS Manager
FROM employees e
INNER JOIN employees m
ON e.manager_id = m.emp_id;
```

🔍 Step-by-Step Explanation

Step 1️⃣ — Table Aliases

- e → represents the **employee**
- m → represents the **manager**
  Both refer to the same table (employees).

Step 2️⃣ — Join Condition

We use:
e.manager_id = m.emp_id
This means:
"Find the row in the same table where the employee's manager_id matches another employee's emp_id."

Step 3️⃣ — MySQL matches rows

Employee (e.emp_name)	e.manager_id	Manager (m.emp_id)	Match?
John	NULL	—	❌
Sara	1	1	✅
Raj	1	1	✅
Meena	2	2	✅
Amit	2	2	✅

✅ Final Output

Employee	Manager
Sara	John
Raj	John
Meena	Sara
Amit	Sara

🧠 Key Points

Concept	Description
Same Table	Table joins with itself
Aliases	Needed to differentiate between the two instances
Join Condition	Usually compares a column to another column in the same table
Use Case	Hierarchical data (employees, categories, family trees, etc.)

INDEX IN MYSQL

An **index** in MySQL is like an **index in a book** — it helps the database find data **faster** without scanning the entire table.
When you run a query with a WHERE, JOIN, or ORDER BY clause, MySQL uses indexes to quickly locate rows.

🧩 **Example Table: employees**
Let's say we have a table like this:

```
CREATE TABLE employees (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(100),
   department VARCHAR(50),
   salary INT,
   city VARCHAR(50)
);
```

And we have **50,000 records** in this table.

🔹 **Case 1: Without Index**

Now, let's say we run this query:

```
SELECT * FROM employees WHERE city = 'Delhi';
```

➡️ **What happens internally:**

- MySQL starts reading from the **first row**.
- It checks each row's city value.
- If it matches 'Delhi', it keeps it.
- It continues till the **last row**.

This is called a **Full Table Scan**.
It's slow because MySQL checks **every single record**, even if only a few match.

🔹 **Case 2: Adding an Index**
Now, we create an index:

```
CREATE INDEX idx_city ON employees(city);
```

➡️ **What happens now:**

- MySQL builds a **sorted data structure (B-Tree)** for the city column.
- Each entry in the index has the **city value** and a **pointer** to the actual row.

So when we run:

```
SELECT * FROM employees WHERE city = 'Delhi';
```

MySQL:

1. Goes to the idx_city index.
2. Quickly finds 'Delhi' using the **B-Tree search** (like binary search).
3. Retrieves only those specific rows from the main table.

✅ **Result:** Query runs much faster — no full scan needed.

🔹 **Case 3: Composite Index (Multiple Columns)**

Suppose we often run queries like:

```
SELECT * FROM employees
WHERE department = 'HR' AND city = 'Delhi';
```

We can create a **composite index**:

```
CREATE INDEX idx_dept_city ON employees(department, city);
```

➡️ This index sorts data **first by department**, then **by city** within each department.
Now MySQL can quickly locate "Delhi" employees **within HR only**, not across the whole table.

🔹 **Case 4: Checking Index Usage**

You can check if your query is using the index:

```
EXPLAIN SELECT * FROM employees WHERE city = 'Delhi';
```

It will show output like:

id	select_type	table	type	possible_keys	key	rows	Extra
1	SIMPLE	employees	ref	idx_city	idx_city	15	Using index condition

🔹 **Types of Indexes in MySQL**

Type	Description
PRIMARY INDEX	Automatically created on the primary key column. Unique and not nullable.
UNIQUE INDEX	Ensures all values in the indexed column are unique.
INDEX (Non-Unique)	Speeds up searches but allows duplicate values.
FULLTEXT INDEX	Used for text search in large text fields (CHAR, VARCHAR, TEXT).
COMPOSITE INDEX	Index on multiple columns (e.g., INDEX (dept_id, emp_name)), useful for combined filtering.

VIEWS IN MYSQL

A **View** is a **virtual table** that is based on the result of an SQL query.
It **does not store data itself** — it just displays data from one or more tables.
You can think of a **View** as a **saved SQL query** that you can reuse like a table.

🧩 **Example**
Suppose we have a table:

```
CREATE TABLE employees (
   emp_id INT,
   emp_name VARCHAR(100),
   department VARCHAR(50),
   salary INT,
   city VARCHAR(50)
);
```

Now you want to see only employees from the HR department frequently.
Instead of writing the query again and again:

```
SELECT emp_id, emp_name, salary
FROM employees
WHERE department = 'HR';
```

You can **create a View** for it 👇

🔹 **Creating a View**

```
CREATE VIEW hr_employees AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE department = 'HR';
```

✅ Now you can use it like a normal table:

```
SELECT * FROM hr_employees;
```

This will show only HR employees.

🔹 **Why Use Views?**

✅ **Advantages**

1. **Simplify complex queries**
    - Save and reuse frequently used SQL queries.
2. **Security**
    - You can hide sensitive columns by giving access only to the view.
3. **Data abstraction**
    - Users can see summarized or filtered data instead of full tables.
4. **Consistency**

- Same logic reused across multiple queries or reports.

Types of Views in MySQL

Views can be categorized based on different criteria. Here are the main types:

1. Simple View

- Based on a single table
- No complex operations like joins, aggregations, or subqueries
- Usually updatable (allows INSERT, UPDATE, DELETE)

```
CREATE VIEW active_employees AS
SELECT emp_id, emp_name, department, salary
FROM employees
WHERE status = 'Active';

-- You can update through this view
UPDATE active_employees SET salary = 55000 WHERE emp_id = 101;
```

2. Complex View

- Involves multiple tables (joins)
- Contains aggregate functions, GROUP BY, DISTINCT, or subqueries
- Usually read-only (not updatable)

```
CREATE VIEW department_summary AS
SELECT d.dept_name, COUNT(e.emp_id) AS employee_count, AVG(e.salary) AS avg_salary
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id  GROUP BY d.dept_name;
```

SUBQUERY

A **Subquery** (also called an **Inner Query** or **Nested Query**) is a query **inside another query**.
It allows you to use the **result of one query** as input to **another query**.

👉 In simple words:
A **subquery** is a **query within a query**.

🧩 **Basic Example**

Let's say we have a table called employees:

emp_id	emp_name	department	salary
1	Alice	HR	45000
2	Bob	IT	70000
3	Carol	IT	60000
4	David	HR	50000

Now, if we want to find **employees who earn more than the average salary**, we can do it using a subquery 👇

```
SELECT emp_name, salary
FROM employees
WHERE salary > (
   SELECT AVG(salary) FROM employees
);
```

➡️ The inner query (SELECT AVG(salary) FROM employees) runs first
➡️ It returns a **single value** (average salary)
➡️ The outer query uses that value to filter employees

🔹 **Structure of a Subquery**

```
SELECT column_name
FROM table
WHERE expression OPERATOR (
   SELECT column_name
   FROM table
   WHERE condition
);
```

Examples of operators:
=, >, <, IN, ANY, ALL, EXISTS

🔹 **Types of Subqueries**

MySQL supports **three main types**:

Type	Description
Single-row subquery	Returns only one value (one row, one column)

Multi-row subquery	Returns multiple rows
Multi-column subquery	Returns multiple columns (used in tuple comparison)

Let's see each one 👇

🧩 1️⃣ **Single-Row Subquery**

Returns only **one value**.

✅ **Example:**

Find employees whose salary equals the **maximum salary** in the company.

```
SELECT emp_name, salary
FROM employees
WHERE salary = (
  SELECT MAX(salary)
  FROM employees
);
```

Here, the subquery returns a **single value** — the highest salary.

🧩 2️⃣ **Multi-Row Subquery**

Returns **multiple rows** — use operators like IN, ANY, or ALL.

✅ **Example (Using IN):**

Find employees who work in the **same department** as employees earning more than 60,000.

```
SELECT emp_name, department
FROM employees
WHERE department IN (
  SELECT department
  FROM employees
  WHERE salary > 60000
);
```

➡️ The subquery returns a list of departments

➡️ The outer query finds all employees in those departments

✅ **Example (Using ANY / ALL):**

Find employees whose salary is **greater than all HR employees' salaries**.

```
SELECT emp_name, salary
FROM employees
WHERE salary > ALL (
  SELECT salary
  FROM employees
  WHERE department = 'HR'
);
```

🧩 3️⃣ Multi-Column Subquery

Returns **multiple columns**, usually compared using a tuple.

✅ **Example:**

Find employees whose **(department, salary)** pair matches another employee's data.

```
SELECT emp_name, department, salary
FROM employees
WHERE (department, salary) IN (
   SELECT department, salary
   FROM employees
   WHERE emp_name = 'Bob'
);
```

🔹 4️⃣ Correlated Subquery

A **Correlated Subquery** is a subquery that **depends on the outer query**.

It runs **once for every row** of the outer query.

✅ **Example:**

Find employees whose salary is **greater than the average salary of their department**.

```
SELECT e1.emp_name, e1.department, e1.salary
FROM employees e1
WHERE e1.salary > (
   SELECT AVG(e2.salary)
   FROM employees e2
   WHERE e2.department = e1.department
);
```

💡 Here:

- The inner query uses a column (e1.department) from the **outer query**
- The inner query executes **for each department** in the outer query

---

🔹 5️⃣ Subquery in the FROM Clause (Inline View)

You can use a subquery **as a temporary table**.

✅ **Example:**

Find the average salary per department and show only departments with an average salary > 50,000.

```
SELECT department, avg_salary
FROM (
   SELECT department, AVG(salary) AS avg_salary
   FROM employees
   GROUP BY department
) AS dept_avg
WHERE avg_salary > 50000;
```

➡️ The inner query creates a virtual table (dept_avg)

➡️ The outer query filters it

🧠 **Summary Table**

Type	Returns	Used With	Example Operator
Single-row	One row	Comparison	=, >, <
Multi-row	Multiple rows	Set comparison	IN, ANY, ALL
Multi-column	Multiple columns	Tuple comparison	IN ((a,b),...)
Correlated	Depends on outer query	Repeated for each row	EXISTS, comparisons
From-clause subquery	Virtual table	Used in reports	FROM (SELECT...)

CTE

A **CTE (Common Table Expression)** is a **temporary result set** that you can reference **within a single SQL query**. It's like creating a **temporary named table** (similar to a view), but it exists **only for that query**.

👉 Think of a CTE as:
"A query inside a query, but with a name — and much cleaner to read."

🧩 Basic Syntax

```
WITH cte_name AS (
   SELECT column1, column2
   FROM table_name
   WHERE condition
)
SELECT *
FROM cte_name
WHERE some_condition;
```

🔹 Example 1: Simple CTE

Let's say we have a table called employees:

emp_id	emp_name	department	salary
1	Alice	HR	45000
2	Bob	IT	70000
3	Carol	IT	60000
4	David	HR	50000

✅ Task:

Find employees who earn **more than the average salary**.

🔸 Using a CTE:

```
WITH avg_salary AS (
   SELECT AVG(salary) AS avg_sal
   FROM employees
)
SELECT emp_name, salary
FROM employees, avg_salary
WHERE employees.salary > avg_salary.avg_sal;
```

OutPut

```
emp_name | salary
-----------------
Bob      | 70000
Carol    | 60000
```

How it works:

1. The WITH clause creates a temporary result (avg_salary).
2. The main query uses that result to filter employees.

🔹 **Example 2: CTE with Multiple Subqueries**

You can define **multiple CTEs** separated by commas.

```
WITH
high_salary AS (
   SELECT emp_id, emp_name, salary
   FROM employees
   WHERE salary > 60000
),
hr_employees AS (
   SELECT emp_id, emp_name, NULL AS salary
   FROM employees
   WHERE department = 'HR'
)
SELECT emp_id, emp_name, salary
FROM high_salary
UNION
SELECT emp_id, emp_name, salary
FROM hr_employees;
```

🎯 **Final Output**

emp_id	emp_name	salary
1	Alice	NULL
2	Bob	70000
4	David	NULL

✅ Here, two CTEs are created (high_salary and hr_employees)
Then they are combined using UNION.

🔸 **Example 3: Recursive CTE (Advanced)**

A Recursive CTE is used when a query needs to refer to itself — like handling hierarchical data (e.g., employee-manager relationships or tree structures).

🧩 Example Table: employees

emp_id	emp_name	manager_id
1	Alice	NULL
2	Bob	1
3	Carol	2
4	David	2

✅ **Goal:**
Find all employees under manager **Alice** (manager_id = 1)

```
WITH RECURSIVE employee_hierarchy AS (
   -- Base case
   SELECT emp_id, emp_name, manager_id
   FROM employees
   WHERE manager_id IS NULL  -- Alice, top-level manager

   UNION ALL

   -- Recursive case
   SELECT e.emp_id, e.emp_name, e.manager_id
   FROM employees e
   INNER JOIN employee_hierarchy eh
   ON e.manager_id = eh.emp_id
)
SELECT * FROM employee_hierarchy;
```

🎯 **Final Output**

```
emp_id | emp_name | manager_id
--------------------------------
1      | Alice    | NULL
2      | Bob      | 1
3      | Carol    | 2
4      | David    | 2
```

How it works:

1. The **base query** selects the top-level manager (Alice).
2. The **recursive part** keeps finding employees whose manager_id is in the previous result.
3. It repeats until no more rows match.

🔹 **Example 4: Using CTE for Readability**
Without a CTE, long queries can look messy 👇

```
SELECT department, AVG(salary)
FROM (
   SELECT * FROM employees WHERE salary > 50000
) AS temp
GROUP BY department;
```

✅ With a CTE, it's cleaner:

```
WITH filtered_emp AS (
   SELECT * FROM employees WHERE salary > 50000
)
SELECT department, AVG(salary)
FROM filtered_emp  GROUP BY department;
```

🔹 **CTE vs Subquery vs View**

Feature	**CTE**	**Subquery**	**View**
Defined with	WITH	Inside query	CREATE VIEW
Lifetime	Only during query execution	Only inside the parent query	Stored permanently
Readability	✅ Very clean	❌ Can be messy	✅ Clean but permanent
Reusable in the same query	✅ Yes	❌ No	✅ Yes
Recursive support	✅ Yes	❌ No	❌ No

STORED PROCEDURES

🔹 What is a Stored Procedure?

A **Stored Procedure** is a **set of SQL statements** that are **stored in the database** and can be **executed repeatedly** as a single unit.

👉 In simple words:

A **Stored Procedure** is like a **function** in programming — it performs a specific task and can accept **inputs**, return **outputs**, and execute **multiple SQL commands** together.

🔹 Why Use Stored Procedures?

Benefit	Description
Reusability	Write once, use multiple times
Performance	Precompiled and executed faster
Security	You can give users access to a procedure without exposing raw tables
Maintainability	Business logic is stored inside the database
Reduced Network Load	Multiple SQL statements execute in one call

Syntax

```
DELIMITER $$

CREATE PROCEDURE procedure_name (parameters)
BEGIN
   -- SQL statements
END $$

DELIMITER ;
```

🔸 Why DELIMITER?

By default, MySQL treats; as the end of a command.
DELIMITER $$ temporarily changes that, so MySQL knows the procedure continues until $$.

🔹 **Example 1: Simple Stored Procedure (No Parameters)**

Let’s create a procedure that shows all employees.

```
DELIMITER $$

CREATE PROCEDURE GetAllEmployees()
BEGIN
   SELECT * FROM employees;
END $$

DELIMITER ;
```

✅ **Execute it:**

```
CALL GetAllEmployees();
```

➡️ This will display all rows from the employees table.

🔹 **Example 2: Stored Procedure with Input Parameter**

You can pass input values (like arguments in a function).

```
DELIMITER $$

CREATE PROCEDURE GetEmployeesByDept(IN dept_name VARCHAR(50))
BEGIN
   SELECT emp_id, emp_name, salary
   FROM employees
   WHERE department = dept_name;
END $$

DELIMITER ;
```

✅ **Execute it:**

```
CALL GetEmployeesByDept('HR');
```

➡️ This shows all employees in the HR department.

🔹 **Example 3: Stored Procedure with Output Parameter**

You can return a single value using an **OUT parameter**.

```
DELIMITER $$

CREATE PROCEDURE GetEmployeeCountByDept(
   IN dept_name VARCHAR(50),
   OUT emp_count INT
)
BEGIN
   SELECT COUNT(*) INTO emp_count
   FROM employees
   WHERE department = dept_name;
END $$

DELIMITER ;
```

✅ **Execute it:**

```
CALL GetEmployeeCountByDept('IT', @count);

SELECT @count;
```

➡️ The procedure stores the result in a variable @count, and then you display it.

🔹 **Example 4: Stored Procedure with INOUT Parameter**

An **INOUT** parameter works as both **input** and **output**.

```
DELIMITER $$

CREATE PROCEDURE IncreaseSalary(INOUT sal INT)
BEGIN
   SET sal = sal + 5000;
END $$

DELIMITER ;
```

✅ **Execute it:**

```
SET @mysal = 45000;

CALL IncreaseSalary(@mysal);

SELECT @mysal;
```

➡️ Output will be **50000**.

🔹 **Example 5: Using IF, CASE, and Loops inside a Procedure**

You can include logic like in programming languages.

```
DELIMITER $$

CREATE PROCEDURE CheckSalary(IN empId INT)
BEGIN
   DECLARE empSal INT;

   SELECT salary INTO empSal
   FROM employees
   WHERE emp_id = empId;

   IF empSal > 60000 THEN
      SELECT 'High salary' AS Result;
   ELSE
      SELECT 'Average or low salary' AS Result;
   END IF;
END $$

DELIMITER ;
```

✅ **Execute:**

```
CALL CheckSalary(2);
```

WINDOW FUNCTIONS

Window functions in **MySQL** are powerful tools that allow you to perform calculations **across a set of rows related to the current row**, without actually grouping or reducing the number of rows (unlike GROUP BY).

They were introduced in **MySQL 8.0** and are extremely useful for tasks like ranking, running totals, moving averages, and percentiles.

- **Basic Syntax**

```
function_name(expression) OVER (
  [PARTITION BY partition_expression]
  [ORDER BY order_expression]
  [frame_clause]
)
```

- **Key Components**

1. **Function** – The window function you're using (e.g., ROW_NUMBER(), SUM(), AVG(), etc.)
2. **PARTITION BY** – Divides rows into groups (like GROUP BY, but without collapsing them).
3. **ORDER BY** – Defines the logical order of rows within each partition.
4. **Frame Clause** *(optional)* – Defines which rows to include in the window for each calculation (e.g., last 3 rows, previous row, etc.).

Types of Window Functions

Category	Example Functions	Common Uses
Ranking	ROW_NUMBER, RANK, DENSE_RANK	Rankings, leaderboards
Aggregates	SUM, AVG, COUNT, MIN, MAX	Running totals, moving averages
Value	LAG, LEAD, FIRST_VALUE, LAST_VALUE	Compare current vs previous rows

- **Key Points**

- Window functions are computed after the WHERE, GROUP BY, and HAVING clauses but before ORDER BY.
- You cannot use window functions in the WHERE clause.
- You can use them in the SELECT and ORDER BY clauses.

🎯 What are Ranking Window Functions?

Ranking window functions assign a **rank or number** to each row based on a specific **order**, optionally divided into **partitions (groups)**.

They **don't group or remove rows**, unlike GROUP BY.

📘 **Sample Table: employees**

Let's assume we have this table:

emp_id	emp_name	department	salary
1	Rahul	Sales	60000
2	Priya	Sales	55000
3	Arjun	Sales	55000
4	Meena	IT	70000
5	Kiran	IT	65000
6	Ravi	IT	65000
7	Sneha	HR	50000

🔢 **1. ROW_NUMBER()**

Purpose: Assigns a unique number to each row — even if two rows have the same values.

Query:

```
SELECT
   emp_name,
   department,
   salary,
   ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num
FROM employees;
```

Result:

emp_name	department	salary	row_num
Rahul	Sales	60000	1
Priya	Sales	55000	2
Arjun	Sales	55000	3
Meena	IT	70000	1
Kiran	IT	65000	2
Ravi	IT	65000	3
Sneha	HR	50000	1

✅ **Note:** Each employee gets a unique number (2 and 3 differ even though the salary is the same).

🔢 2. RANK()

Purpose: Assigns ranks based on order, but **ties get the same rank**, and the **next rank is skipped**.
Query:

```
SELECT
   emp_name,
   department,
   salary,
   RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_num
FROM employees;
```

Result:

emp_name	department	salary	rank_num
Rahul	Sales	60000	1
Priya	Sales	55000	2
Arjun	Sales	55000	2
Meena	IT	70000	1
Kiran	IT	65000	2
Ravi	IT	65000	2
Sneha	HR	50000	1

✅ **Note:** Priya and Arjun both have rank 2, but the next rank is **4** (2 + 2).

🔢 3. DENSE_RANK()

Purpose: Similar to RANK(), but **no ranks are skipped** after a tie.
Query:

```
SELECT
   emp_name,
   department,
   salary,
   DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dense_rank_num
FROM employees;
```

Result:

emp_name	department	salary	dense_rank_num
Rahul	Sales	60000	1
Priya	Sales	55000	2
Arjun	Sales	55000	2
Meena	IT	70000	1
Kiran	IT	65000	2
Ravi	IT	65000	2
Sneha	HR	50000	1

✅ **Note:** Priya and Arjun both have rank 2, but the next rank is **3** (not skipped).

🧱 4. NTILE(n)

Purpose: Divides the result set into **n equal parts (buckets)** and assigns each row to one.
Query:

```
SELECT
  emp_name,
  department,
  salary,
  NTILE(2) OVER (PARTITION BY department ORDER BY salary DESC) AS bucket
FROM employees;
```

Result:

emp_name	department	salary	bucket
Rahul	Sales	60000	1
Priya	Sales	55000	2
Arjun	Sales	55000	2
Meena	IT	70000	1
Kiran	IT	65000	2
Ravi	IT	65000	2
Sneha	HR	50000	1

✅ **Note:** Each department's employees are divided into 2 buckets.

🧩 Summary Table

Function	Ties Handling	Rank Gaps	Use Case
ROW_NUMBER()	No	Always unique	When you just need unique numbering
RANK()	Same rank for ties	Yes (gaps)	For competition-style ranking
DENSE_RANK()	Same rank for ties	No (no gaps)	For compact ranking
NTILE(n)	Group rows into n buckets	N/A	For quartiles, deciles, etc.

🎯 What Are Aggregate Window Functions?

In SQL, **aggregate functions** like SUM(), AVG(), COUNT(), MIN(), MAX() summarize data —
but with **window functions**, they can do that **without collapsing rows**.

This means you can calculate **running totals, averages, and group statistics** while still keeping each individual row in the output.

📘 Sample Table: sales

sale_id	customer_id	order_date	amount
1	C001	01-01-2024	1000
2	C001	05-01-2024	2000
3	C001	10-01-2024	1500
4	C002	03-01-2024	3000
5	C002	08-01-2024	2500
6	C003	02-01-2024	4000

🧮 1. SUM() — Running Total

Goal: Show how each customer's total sales accumulate over time.

Query:

```
SELECT
  customer_id,
  order_date,
  amount,
  SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS running_total
FROM sales;
```

customer_id	order_date	amount	running_total
C001	01-01-2024	1000	1000
C001	05-01-2024	2000	3000
C001	10-01-2024	1500	4500
C002	03-01-2024	3000	3000
C002	08-01-2024	2500	5500
C003	02-01-2024	4000	4000

✅ Explanation:

For each customer_id, MySQL adds up amounts **in date order** — showing how their total sales increase over time.

🔢 2. AVG() — Running Average

Goal: Find the average sale amount per customer as of each transaction.

Query:

```
SELECT
  customer_id,
  order_date,
  amount,
  AVG(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS running_avg
FROM sales;
```

Result:

customer_id	order_date	amount	running_avg
C001	01-01-2024	1000	1000
C001	05-01-2024	2000	1500
C001	10-01-2024	1500	1500
C002	03-01-2024	3000	3000
C002	08-01-2024	2500	2750
C003	02-01-2024	4000	4000

✅ **Explanation:**

Average recalculates after every order — helpful for identifying spending patterns.

🔢 3. COUNT() — Count of Transactions per Customer

Goal: Show how many orders each customer has placed so far.

Query:

```
SELECT
  customer_id,
  order_date,
  COUNT(order_id) OVER (PARTITION BY customer_id ORDER BY order_date) AS order_count
FROM sales;
```

Result:

customer_id	order_date	order_count
C001	01-01-2024	1
C001	05-01-2024	2
C001	10-01-2024	3
C002	03-01-2024	1
C002	08-01-2024	2
C003	02-01-2024	1

✅ **Explanation:**

The COUNT() function counts how many rows (orders) have appeared so far per customer.

🧮 4. MIN() and MAX() — First and Highest Order Value per Customer

Goal: Show the smallest and largest order values per customer.

Query:

```
SELECT
  customer_id,
  order_date,
  amount,
  MIN(amount) OVER (PARTITION BY customer_id) AS min_order,
  MAX(amount) OVER (PARTITION BY customer_id) AS max_order
FROM sales;
```

Result:

customer_id	order_date	amount	min_order	max_order
C001	01-01-2024	1000	1000	2000
C001	05-01-2024	2000	1000	2000
C001	10-01-2024	1500	1000	2000
C002	03-01-2024	3000	2500	3000
C002	08-01-2024	2500	2500	3000
C003	02-01-2024	4000	4000	4000

✅ **Explanation:**

You can use these to compare the current order with the customer's overall lowest or highest.

🧮 5. Using Frame Clauses — Moving Average (Last 2 Orders)

Goal: Find a 3-order moving average per customer (current + 2 previous orders).

Query:

```
SELECT
  customer_id,
  order_date,
  amount,
  AVG(amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS moving_avg
FROM sales;
```

Result:

customer_id	order_date	amount	moving_avg
C001	01-01-2024	1000	1000
C001	05-01-2024	2000	1500
C001	10-01-2024	1500	1500
C002	03-01-2024	3000	3000
C002	08-01-2024	2500	2750
C003	02-01-2024	4000	4000

✅ **Explanation:**

The moving average recalculates considering only the last 3 records (2 before + current).

🎯 What Are Value Window Functions?

They allow you to **access data from other rows** in the same result set — such as **previous, next, first, or last** rows — **without using joins or subqueries**.

Value window functions let you **compare each row to other rows** in the result set.
They are often used to:

- Compare current and previous (or next) row values
- Find trends (increase/decrease)
- Capture first or last values in a group
- Calculate differences or growth rates

📘 Common Value Window Functions

Function	Purpose
LAG(expr [, offset, default])	Access a previous row's value
LEAD(expr [, offset, default])	Access a next row's value
FIRST_VALUE(expr)	Get the first value in the window
LAST_VALUE(expr)	Get the last value in the window

📘 **Sample Table: sales**

id	customer_id	order_date	amount
1	101	01-01-2024	2000
2	101	10-01-2024	3000
3	101	20-01-2024	2500
4	102	05-01-2024	1500
5	102	15-01-2024	1800

⭐ 1. LAG() → Get Previous Row Value

Query

```
SELECT
   customer_id,
   order_date,
   amount,
   LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_amount
FROM sales;
```

Output

customer_id	order_date	amount	prev_amount
101	01-01-2024	2000	NULL
101	10-01-2024	3000	2000
101	20-01-2024	2500	3000
102	05-01-2024	1500	NULL
102	15-01-2024	1800	1500

🔍 **Explanation**

- For each customer, we look at their previous order.
- First order has no previous order → NULL
- Second order of customer 101 → previous amount = 2000
- Third order of customer 101 → previous amount = 3000

⭐ 2. LEAD() → Get Next Row Value

Query

```
SELECT
   customer_id,
   order_date,
   amount,
   LEAD(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS next_amount
FROM sales;
```

Output

customer_id	amount	next_amount
101	2000	3000
101	3000	2500
101	2500	NULL
102	1500	1800
102	1800	NULL

🔍 **Explanation**

- Looks at the next row instead of the previous.

---

3. FIRST_VALUE() → First Value in the Window

Query

```
SELECT
   customer_id,
   order_date,
   amount,
   FIRST_VALUE(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS first_amount
FROM sales;
```

Output

customer_id	order_date	amount	first_amount
101	01-01-2024	2000	2000
101	10-01-2024	3000	2000
101	20-01-2024	2500	2000
102	05-01-2024	1500	1500
102	15-01-2024	1800	1500

🔍 **Explanation**

- For customer 101 → first amount = **2000** (always the same for that customer)
- For customer 102 → first amount = **1500**

⭐ 4. LAST_VALUE() → Last Value in the Window

⚠️ To get the correct last value, you must define the window fully:

Query

```
SELECT
   customer_id,
   order_date,
   amount,
   LAST_VALUE(amount) OVER (
      PARTITION BY customer_id
      ORDER BY order_date
      ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
   ) AS last_amount
FROM sales;
```

Output

customer_id	amount	last_amount
101	2000	2500
101	3000	2500
101	2500	2500

102	1500	1800
102	1800	1800

🔍 **Explanation**

- Customer 101 → last amount = **2500**
- Customer 102 → last amount = **1800**

⭐ **5. NTH_VALUE() → Nth Value in the Window**

Example: 2nd order amount for each customer.

Query

```
SELECT
  customer_id,
  order_date,
  amount,
  NTH_VALUE(amount, 2) OVER (
    PARTITION BY customer_id
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS second_order_amount
FROM sales;
```

Output

customer_id	amount	second_order_amount
101	2000	3000
101	3000	3000
101	2500	3000
102	1500	1800
102	1800	1800

🔍 **Explanation**

- For customer 101 → 2nd value = **3000**
- For customer 102 → 2nd value = **1800**

Function	Purpose
LAG(expr [, offset, default])	Access a previous row's value
LEAD(expr [, offset, default])	Access the next row's value
FIRST_VALUE(expr)	Get the first value in the window
LAST_VALUE(expr)	Get the last value in the window

Summary Table

Function	Description	Looks At	Example Use
LAG()	Previous row's value	Past	Compare current vs previous
LEAD()	Next row's value	Future	Predict next step
FIRST_VALUE()	First row's value	Beginning	Find the starting point
LAST_VALUE()	Last row's value	End	Find the most recent value

✅ **Real-World Use Cases**

Function	Use Case
LAG()	Find the daily change in stock prices
LEAD()	Predict next billing or delivery date
FIRST_VALUE()	Get the customer's first purchase value
LAST_VALUE()	Get the latest salary per employee

UNION and UNION ALL

In **MySQL**, both UNION and UNION ALL are used to **combine the results of two or more SELECT queries** into a single result set.
However, there are **important differences** between the two.

🔹 1. UNION

Purpose:
Combines results from multiple queries **and removes duplicate rows** from the final output.

Syntax:

```
SELECT column1, column2 FROM table1
UNION
SELECT column1, column2 FROM table2;
```

Rules:

- Both SELECT statements must have the **same number of columns**.
- The **columns must have compatible data types**.
- The **column names in the result** are taken from the first SELECT query.

🔹 2. UNION ALL

Purpose:
Combines results from multiple queries and keeps all duplicates**.**
Syntax:

```
SELECT column1, column2 FROM table1
UNION ALL
SELECT column1, column2 FROM table2;
```

🧩 Example Scenario

Suppose we have **two tables**:

📜 **Table 1: customers**

customer_id	name	city
1	Rahul	Delhi
2	Sneha	Mumbai
3	Arjun	Hyderabad

📜 **Table 2: suppliers**

supplier_id	name	city
101	Meera	Mumbai
102	Varun	Delhi
103	Karan	Chennai

🟢 **Example 1: Using UNION**

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers;
```

🔍 **Step-by-step Explanation:**

1. The first query selects all cities from customers:
   → Delhi, Mumbai, Hyderabad
2. The second query selects all cities from suppliers:
   → Mumbai, Delhi, Chennai
3. The UNION keyword **combines** both lists and **removes duplicates**.

✅ **Final Output:**

city
Delhi
Mumbai
Hyderabad
Chennai

🔸 **Why are there no duplicates?**

Because UNION automatically filters them out.

🟠 **Example 2: Using UNION ALL**

```
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;
```

🔍 Step-by-step Explanation:

The first query → Delhi, Mumbai, Hyderabad

The second query → Mumbai, Delhi, Chennai

The UNION ALL simply joins them together — it does not remove duplicates.

✅ Final Output:

city
Delhi
Mumbai
Hyderabad
Mumbai
Delhi
Chennai

🔸 **Why do duplicates appear?**

Because both tables have Delhi and Mumbai, and UNION ALL keeps them.

⚙️ **Performance Difference**

- UNION must **sort and compare** all rows to remove duplicates → **slower**
- UNION ALL simply **appends results** → **faster**

So:

- Use **UNION** when you need **unique results**
- Use **UNION ALL** when you need **all records** or want better **performance**

TRIGGERS

🚀 What is a Trigger in MySQL?

A **trigger** is a special stored program in MySQL that **automatically executes** when a particular **event** happens on a table.

A trigger runs **without manually calling it**.

➤ Triggers are used to:

- Maintain audit logs
- Validate data
- Automatically update values
- Prevent unauthorized changes
- Maintain backup tables

🎯 Trigger Events

Triggers can run on two types of events:

1️⃣ BEFORE

Runs *before* the query is executed.

2️⃣ AFTER

Runs *after* the query is executed.

And they run on:

- **INSERT**
- **UPDATE**
- **DELETE**

So total 6 types:

Event	Trigger
BEFORE INSERT	Yes
AFTER INSERT	Yes
BEFORE UPDATE	Yes
AFTER UPDATE	Yes
BEFORE DELETE	Yes
AFTER DELETE	Yes

⭐ Basic Trigger Syntax

```
CREATE TRIGGER trigger_name

BEFORE/AFTER INSERT/UPDATE/DELETE

ON table_name

FOR EACH ROW

BEGIN

   -- trigger logic

END;
```

⭐ **Example 1: Audit Log on INSERT (MOST COMMON)**

✓ **Problem:**

Whenever someone inserts a new employee, we want to store that action into a log table.

Step 1: Create a log table

```
CREATE TABLE employee_log (
   log_id INT AUTO_INCREMENT PRIMARY KEY,
   emp_id INT,
   action_by VARCHAR(50),
   action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Step 2: Create a trigger

```
CREATE TRIGGER after_employee_insert
AFTER INSERT
ON employees
FOR EACH ROW
BEGIN
   INSERT INTO employee_log(emp_id, action_by)
   VALUES (NEW.emp_id, USER());
END;
```

🔍 **Explanation**

- NEW.emp_id → value inserted
- Trigger fires **after** the insert happens
- Adds entry to employee_log automatically

⭐ **Example 2: BEFORE UPDATE Trigger to Validate Salary**

✓ **Problem:**

Salary should not go below 5000. If the user updates salary < 5000 → set it to 5000.

Trigger:

```
CREATE TRIGGER before_salary_update
BEFORE UPDATE
ON employees
FOR EACH ROW
BEGIN
  IF NEW.salary < 5000 THEN
    SET NEW.salary = 5000;
  END IF;
END;
```

🔍 **Explanation**

- BEFORE UPDATE → runs before updating
- NEW.salary → updated value
- If value is invalid → automatically corrects it

⭐ **Example 3: AFTER DELETE Trigger – Store Deleted Records**

✓ **Problem:**

When someone deletes a row in the employees table, store the deleted data in a backup table.

Trigger:

```
CREATE TRIGGER after_employee_delete
AFTER DELETE
ON employees
FOR EACH ROW
BEGIN
  INSERT INTO employees_backup(emp_id, name, salary)
  VALUES (OLD.emp_id, OLD.name, OLD.salary);
END;
```

🔍 Explanation

- OLD.column → value *before delete*
- Trigger keeps a backup of the deleted record

🧠 NEW and OLD Keywords in Triggers

Keyword	Meaning	Used In
NEW	New row values	INSERT, UPDATE
OLD	Existing row before change	UPDATE, DELETE

🎯 Summary of Triggers

Trigger Type	When It Runs	Uses
BEFORE INSERT	Before insert	Validate data, modify insert values
AFTER INSERT	After insert	Log new records, create history
BEFORE UPDATE	Before update	Validate updates, prevent wrong data
AFTER UPDATE	After update	Audit updates
BEFORE DELETE	Before delete	Prevent delete
AFTER DELETE	After delete	Store deleted rows, maintain backup

FUNCTIONS

1️⃣ STRING FUNCTIONS

1. LENGTH(str)

Explanation: Returns the length of the string in **bytes** (ASCII letters = 1 byte each).

```
SELECT LENGTH('SQL');
```

Output:

3

---

2. CHAR_LENGTH(str)

Explanation: Returns length in **characters** (not bytes).

```
SELECT CHAR_LENGTH('Lalit');
```

Output:

5

---

3. CONCAT(str1, str2, …)

Explanation: Joins multiple strings together.

```
SELECT CONCAT('Data', ' ', 'Analyst');
```

Output:

Data Analyst

---

4. CONCAT_WS(separator, str1, str2, …)

Explanation: Concatenates strings with a **separator** between them.

```
SELECT CONCAT_WS('-', '2025', '01', '17');
```

Output:

2025-01-17

---

5. UPPER(str)

Explanation: Converts all characters to uppercase.

```
SELECT UPPER('mysql');
```

Output:

MYSQL

---

6. LOWER(str)

Explanation: Converts all characters to lowercase.

```
SELECT LOWER('MySQL');
```

Output:

mysql

---

7. LEFT(str, n)

Explanation: Returns the first **n** characters from the left.

```
SELECT LEFT('Analytics', 4);
```

Output:

Anal

---

8. RIGHT(str, n)

Explanation: Returns the last **n** characters from the right.

```
SELECT RIGHT('Analytics', 4);
```

Output:

tics

---

9. SUBSTRING(str, start, length)

Explanation: Extracts a substring from a starting position.

```
SELECT SUBSTRING('Analytics', 2, 4);
```

Output:

naly

---

10. REPLACE(str, find, replace)

Explanation: Replaces all occurrences of a substring.

```
SELECT REPLACE('abcabc', 'a', 'A');
```

Output:

AbcAbc

---

11. TRIM(str)

Explanation: Removes leading and trailing spaces.

```
SELECT TRIM('   SQL   ');
```

Output:

SQL

---

12. LPAD(str, length, pad)

Explanation: Pads the string on the left to reach the specified length.

```
SELECT LPAD('7', 3, '0');
```

Output:

007

---

13. RPAD(str, length, pad)

Explanation: Pads the string on the right.

```
SELECT RPAD('7', 3, '0');
```

Output:

700

2 NUMERIC FUNCTIONS

1. ABS(number)

Explanation: Returns the absolute (positive) value.

```
SELECT ABS(-10);
```

Output:

10

---

2. ROUND(number, decimals)

Explanation: Rounds a number to given decimal places.

```
SELECT ROUND(12.567, 2);
```

Output:

12.57

---

3. CEIL(number)

Explanation: Rounds a number **up** to the nearest integer.

```
SELECT CEIL(4.1);
```

Output:

5

---

4. FLOOR(number)

Explanation: Rounds a number **down** to the nearest integer.

```
SELECT FLOOR(4.9);
```

Output:

4

---

5. POWER(a, b)

Explanation: Returns **a** raised to the power **b**.

```
SELECT POWER(2, 3);
```

Output:

8

---

6. SQRT(n)

Explanation: Returns square root of a number.

```
SELECT SQRT(25);
```

Output:

5

---

7. MOD(a, b)

Explanation: Returns the remainder of a ÷ b.

```
SELECT MOD(10, 3);
```

Output:

1

---

8. RAND()

Explanation: Returns a random floating number between 0 and 1.

```
SELECT RAND();
```

Output (example):

0.548233

3️⃣ DATE & TIME FUNCTIONS

Let's assume today = **2025-01-17**.

---

1. CURDATE()

Explanation: Returns the current system date.

```
SELECT CURDATE();
```

Output:

2025-01-17

---

2. CURTIME()

Explanation: Returns the current system time.

```
SELECT CURTIME();
```

Output (example):

12:30:10

---

3. NOW()

Explanation: Returns the current date and time.

```
SELECT NOW();
```

Output:

2025-01-17 12:30:10

---

4. YEAR(date)

Explanation: Extracts the year from a date.

```
SELECT YEAR('2025-11-17');
```

Output:

2025

---

5. MONTH(date)

Explanation: Extracts the month number.

```
SELECT MONTH('2025-11-17');
```

Output:

11

---

6. DAY(date)

Explanation: Extracts the day of the month.

```
SELECT DAY('2025-11-17');
```

Output:

17

---

7. DAYNAME(date)

Explanation: Returns weekday name.

```
SELECT DAYNAME('2025-01-17');
```

Output:

Friday

---

8. DATE_ADD(date, INTERVAL value unit)

Explanation: Adds a time interval to a date.

```
SELECT DATE_ADD('2025-01-01', INTERVAL 10 DAY);
```

Output:

2025-01-11

---

9. DATE_SUB(date, INTERVAL value unit)

Explanation: Subtracts a time interval from a date.

```
SELECT DATE_SUB('2025-01-10', INTERVAL 5 DAY);
```

Output:

2025-01-05

---

10. DATEDIFF(date1, date2)

Explanation: Returns the number of days between two dates.

```
SELECT DATEDIFF('2025-01-20', '2025-01-15');
```

Output:

5

---

11. DATE_FORMAT(date, format)

Explanation: Formats a date in a custom pattern.

```
SELECT DATE_FORMAT('2025-01-17', '%d-%m-%Y');
```

Output:

17-01-2025

4️⃣ TYPE CONVERSION FUNCTIONS

---

1. CAST(value AS type)

Explanation: Converts a value to a specific data type.

```
SELECT CAST('2025-01-01' AS DATE);
```

Output:

2025-01-01

---

2. CONVERT(value, type)

Explanation: Converts a value to the given type (same as CAST).

```
SELECT CONVERT('123', SIGNED);
```

Output:

123

---

3. FORMAT(number, decimals)

Explanation: Formats number with commas.

```
SELECT FORMAT(12345.678, 2);
```

Output:

12,345.68

5️⃣ NULL FUNCTIONS

---

1. IFNULL(a, b)

Explanation: Returns **b** if **a** is NULL.

```
SELECT IFNULL(NULL, 'N/A');
```

Output:

N/A

---

2. NULLIF(a, b)

Explanation: Returns NULL if both values are equal.

```
SELECT NULLIF(10, 10);
```

Output:

NULL

---

3. COALESCE(a, b, c, …)

Explanation: Returns the first non-NULL value.

```
SELECT COALESCE(NULL, NULL, 'Hello');
```

Output:

Hello

6️⃣ JSON FUNCTIONS — Explanation + Example + Output

---

1. JSON_OBJECT(key, value)

Explanation: Creates a JSON object.

```
SELECT JSON_OBJECT('name','Lalit', 'role','Analyst');
```

Output:

{"name":"Lalit","role":"Analyst"}

---

2. JSON_ARRAY(value1, value2, …)

Explanation: Creates a JSON array.

```
SELECT JSON_ARRAY(10, 20, 30);
```

Output:

[10, 20, 30]

---

3. JSON_EXTRACT(json, path)

Explanation: Extracts value from JSON using a path.

```
SELECT JSON_EXTRACT('{"name":"Lalit"}', '$.name');
```

Output:

"Lalit"

---

4. JSON_SET(json, path, value)

Explanation: Replaces/updates a value inside a JSON.

```
SELECT JSON_SET('{"name":"Lalit"}', '$.name', 'Kumar');
```

Output:

{"name":"Kumar"}

✅ **1. What is NULL?**

NULL means **unknown or missing value**.
It is **not 0**, **not an empty string**, **not false**.
It simply means "value is not available".

Example:

value
NULL
10
''

Here:

- NULL = unknown
- '' = empty string
- 10 = number

✅ **2. NULL Comparison Rule**

❌ **You cannot use =, !=, >, < with NULL.**

Example:

```
SELECT NULL = NULL;
```

Output:

NULL

Why?
Because SQL does not know if two unknown values are equal.

✅ 3. Correct way to check NULL

Use:

- **IS NULL**
- **IS NOT NULL**

Example:

```
SELECT
  CASE
    WHEN NULL IS NULL THEN 'Yes'
    ELSE 'No'
  END AS result;
```

Output:

Yes

✅ 4. NULL in Arithmetic Operations

Any arithmetic with NULL = NULL

Example:

```
SELECT 10 + NULL AS sum;
```

Output:

NULL

Because one value is unknown.

✅ 5. NULL in String Concatenation

```
SELECT CONCAT('Hello ', NULL);
```

Output:

NULL

Use **IFNULL** or **COALESCE** to fix.

✅ 6. Important NULL Handling Functions

🔹 6.1 IFNULL(expr, replacement)

Returns *replacement* if expr is NULL.

Example:

```
SELECT IFNULL(NULL, 'No Value') AS result;
```

Output:

No Value

🔹 6.2 COALESCE(expr1, expr2, ...)

Returns the **first non-null value** in the list.

```
SELECT COALESCE(NULL, NULL, 20, 50);
```

Output:

20

🔹 6.3 NULLIF(expr1, expr2)

If both values are equal → returns NULL
Else → returns expr1

Example:

```
SELECT NULLIF(10, 10);
```

Output:

NULL

Another:

```
SELECT NULLIF(10, 5);
```

Output:

10

🔹 6.4 IF(condition, value_if_true, value_if_false)

Example:

```
SELECT IF(NULL IS NULL, 'Null Found', 'Not Null');
```

Output:

```
Null Found
```

🔹 6.5 ISNULL(expr)

Returns **1** if NULL, else **0**

```
SELECT ISNULL(NULL);
```

Output:

```
1
```

✅ 7. NULL in Aggregate Functions

Function	Ignores NULL?	Example
SUM()	✔ ignores	SUM(NULL, 10) = 10
AVG()	✔ ignores	
COUNT(*)	❌ counts null	counts all rows
COUNT(column)	✔ ignores null	

Example:

Table:

value
10
NULL

```
SELECT COUNT(value), COUNT(*), SUM(value)
FROM test;
```

Output:

COUNT(value)	COUNT(*)	SUM(value)
1	2	10

✅ 8. NULL in WHERE Clause

❌ Wrong:

```
WHERE column = NULL;
```

✓ Correct:

```
WHERE column IS NULL;
```

🔥 9. NULL in ORDER BY

NULL always appears:

- **First** → in ASC (default)
- **Last** → in DESC

Example:

value
10
NULL
5

```
SELECT *
FROM t
ORDER BY value ASC;
```

Output:

NULL

5

10

✅ 10. Replace NULL with Default Values

Using IFNULL

```
SELECT IFNULL(address, 'No Address') FROM customers;
```

Using COALESCE

```
SELECT COALESCE(phone, 'Not Provided') FROM customers;
```

🎯 Summary Table

Concept	Result
NULL = NULL	NULL
NULL <> NULL	NULL
NULL + 10	NULL
CONCAT('A', NULL)	NULL
IS NULL	TRUE
IS NOT NULL	TRUE
COALESCE	First non-null value
IFNULL	Replace NULL
NULLIF	Return NULL if equal

LIMIT Clause

The MySQL LIMIT Clause

The `LIMIT` clause is used to specify the number of records to return.

The `LIMIT` clause is useful on large tables with thousands of records. Returning a large number of records can impact performance.

LIMIT Syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition
LIMIT number;
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico

3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

MySQL LIMIT Examples

The following SQL statement selects the first three records from the "Customers" table:

Example

```
SELECT * FROM Customers
LIMIT 3;
```

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	5021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.	5023	Mexico

What if we want to select records 4 - 6 (inclusive)?

MySQL provides a way to handle this: by using OFFSET.

The SQL query below says "return only 3 records, start on record 4 (OFFSET 3)":

Example

```
SELECT * FROM Customers
LIMIT 3 OFFSET 3;
```

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden
6	Blauer See Delikatessen	Hanna Moos	Forsterstr. 57	Mannheim	68306	Germany

ADD a WHERE CLAUSE

The following SQL statement selects the first three records from the "Customers" table, where the country is "Germany":

Example

```
SELECT * FROM Customers
WHERE Country='Germany'
LIMIT 3;
```

ADD an ORDER BY CLAUSE

The following SQL statement sorts the customers by country before returning the first three records from the selection:

Example

```
SELECT * FROM Customers
ORDER BY Country
LIMIT 3;
```

To get the **third-highest salary** in MySQL, you can use **ORDER BY + LIMIT** or **DISTINCT + LIMIT**, or a **subquery with MAX()**.

Below are the **best methods** 👇

✅ **Method 1: Using ORDER BY + LIMIT**

Works when **salaries can repeat** (but you want unique salaries).

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

✓ OFFSET 2 means skip the highest (0), second-highest (1), and return the third (2).

✅ **Method 2: Using Subquery with MAX()**

Get the third highest by excluding the top 2:

```
SELECT MAX(salary) AS third_highest_salary
FROM employees
WHERE salary < (
   SELECT MAX(salary)
   FROM employees
   WHERE salary < (
      SELECT MAX(salary)
      FROM employees
   )
);
```

✓ Good for **interview answers**

✓ Finds the 3rd highest **distinct** salary

✅ **Method 3: Using DENSE_RANK() (MySQL 8+)**

```
SELECT salary
FROM (
   SELECT salary,
        DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
   FROM employees
) AS t
WHERE rnk = 3;
```

✓ Uses window functions

✓ Gives the salary even if duplicates exist

✓ Best for **large datasets**

📝 **Example Table**

employee	salary
A	5000
B	4000
C	6000
D	7000
E	5000

Third-highest distinct salary → 5000